

Week 2: Implementation of Security Measures

1. Introduction

This report describes the implementation of basic security measures to address vulnerabilities identified during the security assessment in Week 1. The objective is to enhance application security through input validation, password protection, authentication mechanisms, and secure HTTP configurations.

2. Input Validation and Sanitization

User input validation is essential to prevent invalid or malicious data from being processed by the application.

Code Implementation

```
function isValidEmail(email) {  
    return email.includes("@") && email.includes(".");  
}  
  
console.log(isValidEmail("test@gmail.com")); // true  
console.log(isValidEmail("abc")); // false
```



JavaScript Online Compiler

main.js	Run	Output
<pre>1- function isValidEmail(email) { 2 return email.includes("@") && email.includes("."); 3 } 4 5 console.log(isValidEmail("test@gmail.com")); // true 6 console.log(isValidEmail("abc")); // false</pre>		<pre>true false === Code Execution Success</pre>

Explanation

The validation function checks whether the email contains "@" and ".", ensuring a basic valid format. This helps prevent incorrect or potentially harmful input.

3. Password Hashing

Passwords should not be stored in plain text. Instead, they must be securely hashed before storage.

Code Example

```
const bcrypt = require('bcrypt');

async function hashPassword(password) {
  const hashedPassword = await bcrypt.hash(password, 10);
  return hashedPassword;
}
```

Explanation

Hashing passwords ensures that even if the database is compromised, attackers cannot retrieve the original passwords.

4. Token-Based Authentication

Token-based authentication is used to securely manage user sessions.

Code Example

```
const jwt = require('jsonwebtoken');

function generateToken(user) {
```

```
const token = jwt.sign({ id: user.id }, 'secret-key');  
return token;  
}
```

Explanation

JSON Web Tokens (JWT) allow secure authentication by generating a token that verifies the user's identity without maintaining server-side sessions.

5. Secure HTTP Headers

Security headers protect the application from common web-based attacks.

Code Example

```
const helmet = require('helmet');  
  
app.use(helmet());
```

Explanation

Helmet.js helps secure the application by automatically setting important HTTP headers such as Content Security Policy, X-Frame-Options, and X-Content-Type-Options.

6. Summary of Improvements

The following security measures were implemented:

- Input validation to prevent invalid or malicious data

- Password hashing for secure credential storage
 - Token-based authentication using JWT
 - Secure HTTP headers using Helmet.js
-

7. Conclusion

The implementation of these security measures enhances the security of the application by reducing the risk of common vulnerabilities such as cross-site scripting, data exposure, and unauthorized access. These practices follow standard security guidelines and improve the overall reliability of the system.